Код класса для тестирования:

```
class AuthManager:
    def __init__(self, connection):
        self.connection = connection
        self.create_tables()

    def create_tables(self):
        with self.connection:
            self.connection.execute("""
                CREATE TABLE IF NOT EXISTS users (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    username TEXT NOT NULL UNIQUE,
                    password TEXT NOT NULL,
                    country TEXT NOT NULL,
                    balance REAL NOT NULL
                )
            """)

    def register_user(self, username, password, country, balance):
        with self.connection:
            self.connection.execute("""
                INSERT INTO users (username, password, country, balance)
                VALUES ('{}', '{}', '{}', {})
            """.format(username, password, country, balance))

    def authenticate_user(self, username, password):
        cursor = self.connection.cursor()
        cursor.execute("""
            SELECT * FROM users
            WHERE username = '{}' AND password = '{}'
        """.format(username, password))
        return cursor.fetchone()

    def delete_user(self, user_id):
        with self.connection:
            self.connection.execute("""
                DELETE FROM users WHERE id = {}
            """.format(user_id))

    def get_user_by_id(self, user_id):
        cursor = self.connection.cursor()
        cursor.execute("""
            SELECT * FROM users WHERE id = {}
        """.format(user_id))
```

```
        return cursor.fetchone()

    def count_users_by_country(self, country):
        cursor = self.connection.cursor()
        cursor.execute("""
            SELECT COUNT(*) FROM users WHERE country = '{}'
        """.format(country))
        return cursor.fetchone()[0]

    def transfer_balance(self, from_user_id, to_user_id, amount):
        with self.connection:
            cursor = self.connection.cursor()
            cursor.execute(
                "SELECT balance FROM users WHERE id =
{}".format(from_user_id)
            )

            from_balance = cursor.fetchone()[0]

            if from_balance < amount:
                raise ValueError("Insufficient funds")

            self.connection.execute("""
                UPDATE users SET balance = balance - {}
                WHERE id = {}
            """.format(amount, from_user_id))

            self.connection.execute("""
                UPDATE users SET balance = balance + {}
                WHERE id = {}
            """.format(amount, to_user_id))
```

Код тестов:

```python
import sqlite3
import pytest
from base_class import AuthManager

# тестики
# фикстуры(базы данных)

@pytest.fixture
def db():
    """Создание временной базы данных SQLite."""
    connection = sqlite3.connect(":memory:")
    yield connection
    connection.close()


@pytest.fixture
def auth_manager(db):
    """Создание экземпляра AuthManager."""
    return AuthManager(db)


# базовые тесты

@pytest.mark.basic
def test_get_user_by_id(auth_manager):
    """Проверка получения информации о пользователе."""

    auth_manager.register_user(
        "alex", "password123", "Latvia", 1500
    )

    user = auth_manager.get_user_by_id(1)

    assert user is not None
    assert user[0] == 1
    assert user[1] == "alex"
    assert user[2] == "password123"
    assert user[3] == "Latvia"
    assert user[4] == 1500


@pytest.mark.basic
def test_authenticate_wrong_password(auth_manager):
    """Проверка отказа в авторизации при неправильном пароле."""
```

```
    auth_manager.register_user(
        "maria", "correct_password", "Estonia", 800
    )

    result = auth_manager.authenticate_user(
        "maria", "wrong_password"
    )

    assert result is None


@pytest.mark.basic
def test_transfer_balance(auth_manager):
    """Проверка изменения балансов двух пользователей."""

    auth_manager.register_user(
        "sender", "pass1", "Latvia", 120
    )

    auth_manager.register_user(
        "receiver", "pass2", "Latvia", 35
    )

    auth_manager.transfer_balance(1, 2, 45)

    sender = auth_manager.get_user_by_id(1)
    receiver = auth_manager.get_user_by_id(2)

    assert sender[4] == 75
    assert receiver[4] == 80


# параметризованные тесты

@pytest.mark.parametrized
@pytest.mark.parametrize(
    "username, password, country, balance",
    [
        ("ivan", "pass111", "Latvia", 100),
        ("elena", "pass222", "Estonia", 500),
        ("pavel", "pass333", "Lithuania", 0)
    ]
)
def test_register_user_parametrized(
    auth_manager, username, password, country, balance
```

```
):
    """Проверка регистрации с различными входными данными."""

    auth_manager.register_user(
        username, password, country, balance
    )

    user = auth_manager.get_user_by_id(1)

    assert user is not None
    assert user[1] == username
    assert user[2] == password
    assert user[3] == country
    assert user[4] == balance


@pytest.mark.parametrized
@pytest.mark.parametrize(
    "user_id, expected_remaining",
    [
        (1, {"user_b", "user_c"}),
        (2, {"user_a", "user_c"}),
        (3, {"user_a", "user_b"})
    ]
)
def test_delete_user_parametrized(
    auth_manager, user_id, expected_remaining
):
    """Проверка удаления различных пользователей."""

    auth_manager.register_user("user_a", "pass", "Latvia", 100)
    auth_manager.register_user("user_b", "pass", "Estonia", 200)
    auth_manager.register_user("user_c", "pass", "Germany", 300)

    auth_manager.delete_user(user_id)

    assert auth_manager.get_user_by_id(user_id) is None

    remaining_users = set()

    for current_id in (1, 2, 3):
        user = auth_manager.get_user_by_id(current_id)

        if user is not None:
            remaining_users.add(user[1])
```

```
    assert remaining_users == expected_remaining


@pytest.mark.parametrized
@pytest.mark.parametrize(
    "country, expected_count",
    [
        ("Latvia", 2),
        ("Estonia", 1),
        ("Lithuania", 0)
    ]
)
def test_count_users_by_country(
    auth_manager, country, expected_count
):
    """Проверка подсчета пользователей по странам."""

    auth_manager.register_user("user1", "pass", "Latvia", 100)
    auth_manager.register_user("user2", "pass", "Latvia", 200)
    auth_manager.register_user("user3", "pass", "Estonia", 300)

    result = auth_manager.count_users_by_country(country)

    assert result == expected_count


# тесты исключений

@pytest.mark.exception
def test_duplicate_username(auth_manager):
    """Проверка запрета регистрации одинаковых логинов."""

    auth_manager.register_user(
        "admin", "password", "Latvia", 1000
    )

    with pytest.raises(sqlite3.IntegrityError):
        auth_manager.register_user(
            "admin", "another_password", "Estonia", 500
        )
@pytest.mark.exception
def test_transfer_insufficient_funds(auth_manager):
    """Проверка исключения при недостаточном балансе."""

    auth_manager.register_user(
        "poor_user", "pass", "Latvia", 50
```

```
    )

    auth_manager.register_user(
        "receiver", "pass", "Estonia", 100
    )

    with pytest.raises(ValueError, match="Insufficient funds"):
        auth_manager.transfer_balance(1, 2, 100)

    sender = auth_manager.get_user_by_id(1)
    receiver = auth_manager.get_user_by_id(2)

    assert sender[4] == 50
    assert receiver[4] == 100


# sql инъекция

@pytest.mark.security
@pytest.mark.xfail(
    strict=True,
    reason="Исходный AuthManager уязвим к SQL-инъекции"
)
def test_sql_injection(auth_manager):
    """Проверка защиты от обхода аутентификации."""

    auth_manager.register_user(
        "admin", "secret_password", "Latvia", 10000
    )

    malicious_password = "' OR '1'='1"

    result = auth_manager.authenticate_user(
        "admin", malicious_password
    )

    assert result is None


# пропускаемый тест

@pytest.mark.skip(reason="Тест временно отключен")
def test_skipped_example():
    """Демонстрация пропуска тестирования."""

    assert False
```

Результаты тестирования

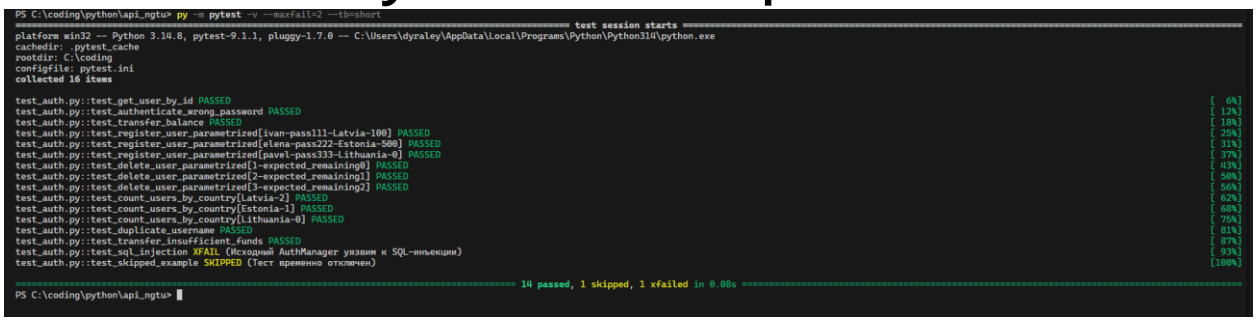


Рисунок 1 – Запуск тестов

Далее делаем в каком-то из тестов преднамеренную ошибку и смотрим на результат.
Буду изменять функцию «test_authenticate_wrong_password», где в result будет передан в качестве второго аргумента правильный пароль вместо ожидаемого неправильного.

```python
@pytest.mark.basic
def test_authenticate_wrong_password(auth_manager):
    """Проверка отказа в авторизации при неправильном пароле."""

    auth_manager.register_user(
        "maria", "correct_password", "Estonia", 800
    )

    result = auth_manager.authenticate_user(
        "maria", "correct_password"
    )

    assert result is None
```

Рисунок 2 – Функция, в которой специально была допущена ошибка

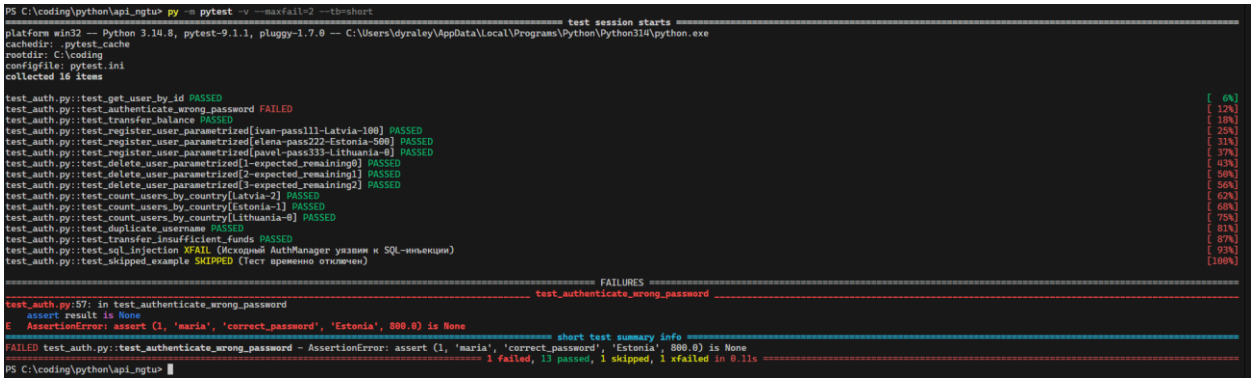


Рисунок 3 – Результаты повторного тестирования